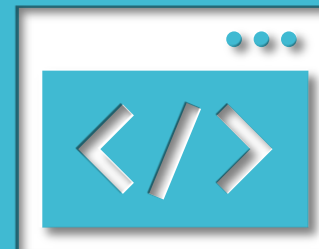


STRUCTURES DE DONNÉES & TYPES ABSTRAITS DE DONNÉES

PARTIE 2 : Listes
Simplement Chaînées



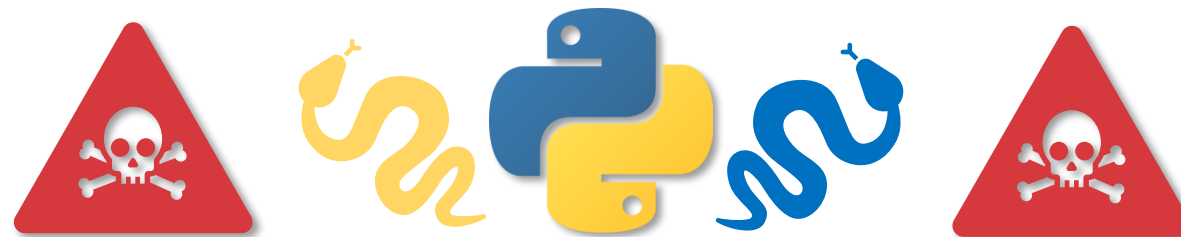
1010
1010

INTRODUCTION

Principe des listes chaînées & comparaisons

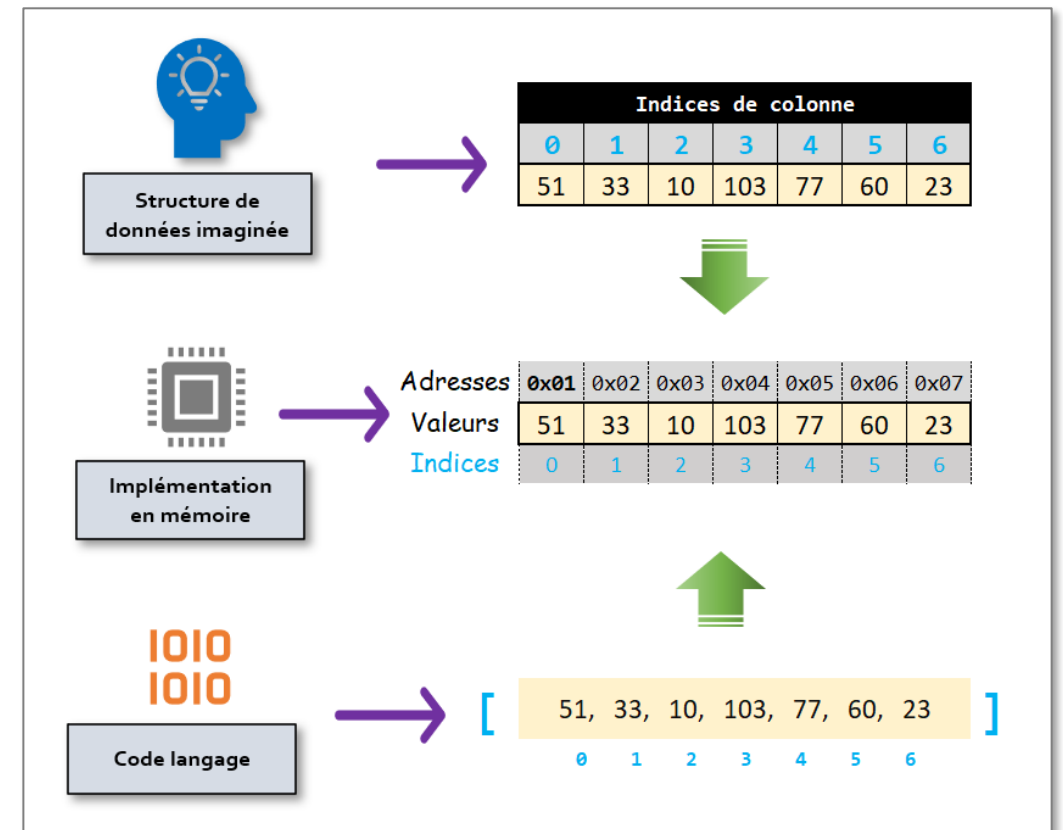
- Rappel sur les listes de Python, tableaux et listes chaînées :
 - En informatique les tableaux et les listes chaînées sont 2 structures de données concurrentes. Elles répondent au même besoin initial :
 - **Stocker une collection de valeurs**
 - **Pouvoir accéder à chaque valeur afin de la lire ou de la modifier.**
 - Mais l'implémentation des 2 structures est fondamentalement **différente !**

- Rappel sur les listes de Python, tableaux et listes chaînées :
 - **Attention**, **Python** nous induit en erreur avec son implémentation des listes qui **N'EST PAS** en réalité une **liste chaînée** !
 - **Les listes** de **Python** en termes d'interface sont plus proches des TABLEAUX.
 - Idem pour l'implémentation en mémoire qui est très commune à l'implémentation des TABLEAUX.
 - Par contre, les liste de **Python** sont dynamiques comme le sont les listes chaînées.



INTRODUCTION

- Rappel sur les tableaux (array) :
 - Un tableau est implémenté par une suite d'espaces/cellules mémoire **CONTIGUS**
 - **TOUS** les espaces/cellules mémoires ont des tailles **IDENTIQUES**.



INTRODUCTION

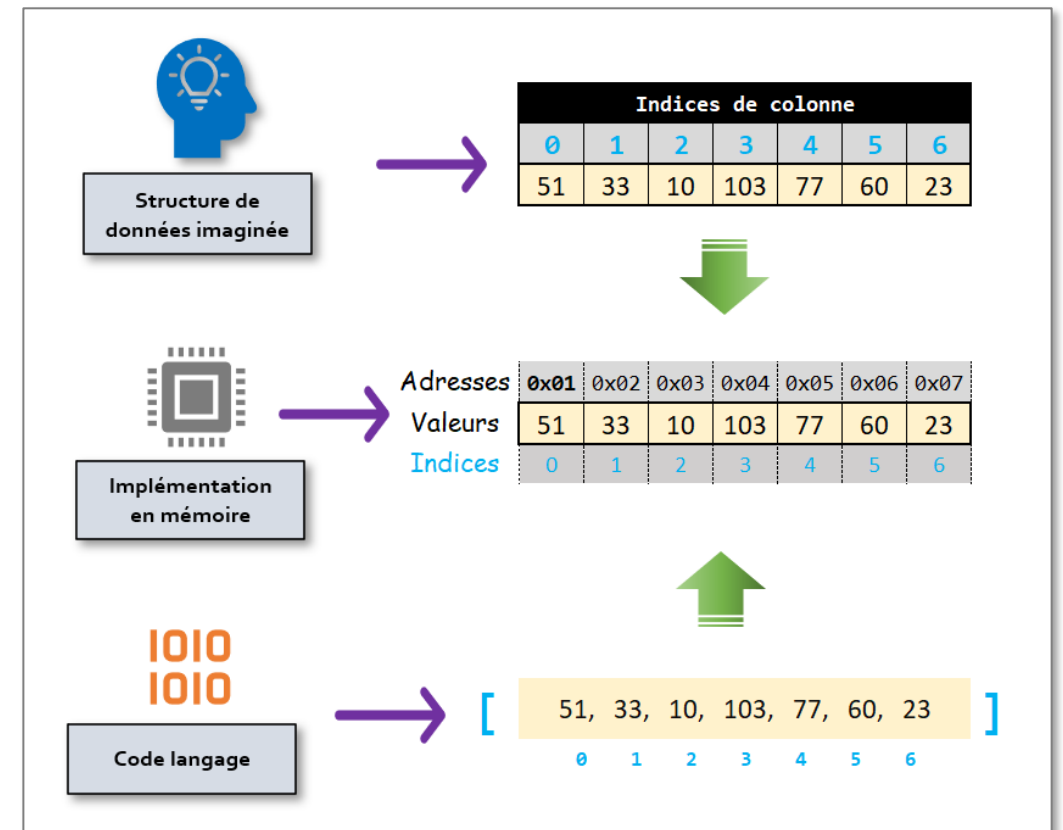
- Rappel sur les tableaux (array) :

- Avantage :

- Un **accès rapide** aux données qui les rend performantes pour les accès en lecture/écriture des valeurs stockées.

- Inconvénient :

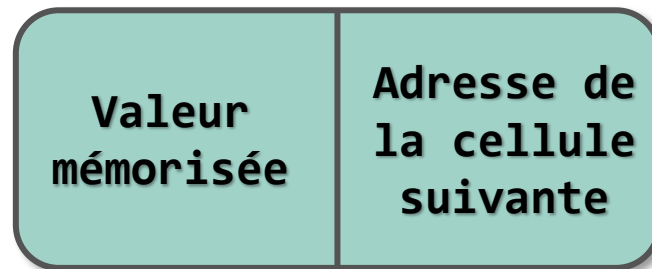
- C'est une structure naturellement **statique**, l'insertion d'une ou plusieurs cellules implique de reconstruire intégralement le tableau !



INTRODUCTION

- **La liste chaînée :**

- L'idée de base est de s'affranchir de la contrainte de contiguïté des cellules mémoires afin de pouvoir insérer/supprimer librement des cellules.
- Pour assurer la cohérence de la liste, chaque cellule contiendra la valeur à mémoriser mais également l'adresse mémoire de la cellule suivante

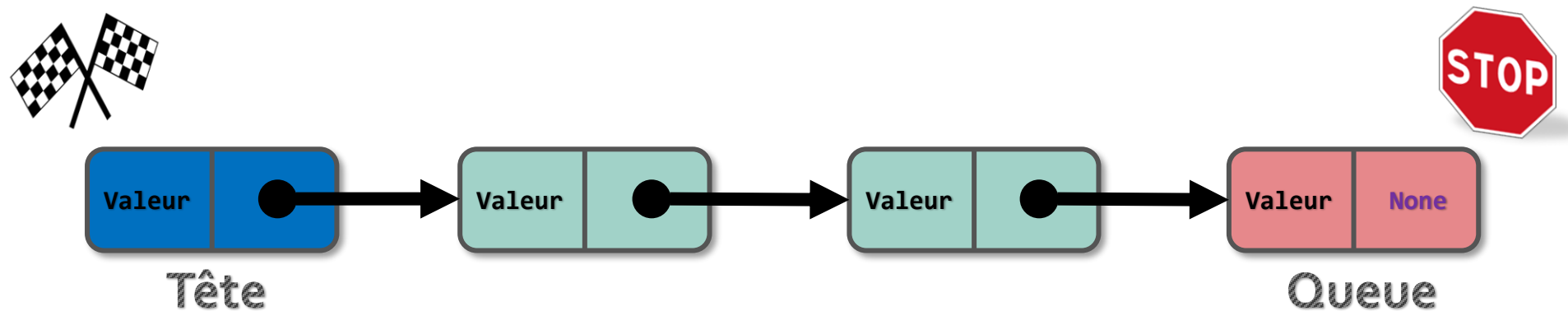


Cellule/maillon

INTRODUCTION

- La liste chaînée :

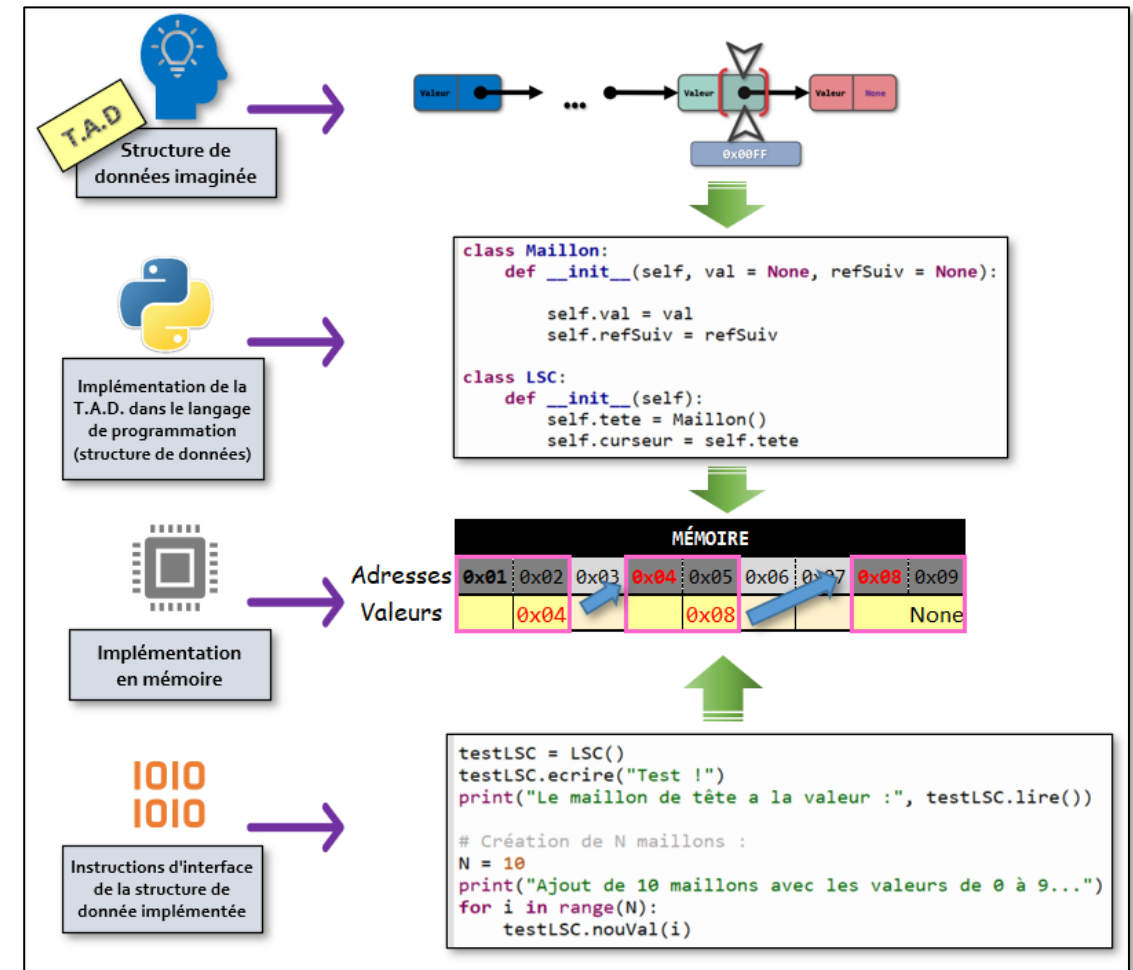
- La 1^{ère} cellule est appelée « cellule de tête » ou « **tête** ».
- La dernière cellule est appelée cellule de queue ou « **queue** ».
- La cellule de queue doit contenir un constante qui permet de la reconnaître, en Python on peut par exemple utiliser la constante **None**.



INTRODUCTION

- La liste chaînée :

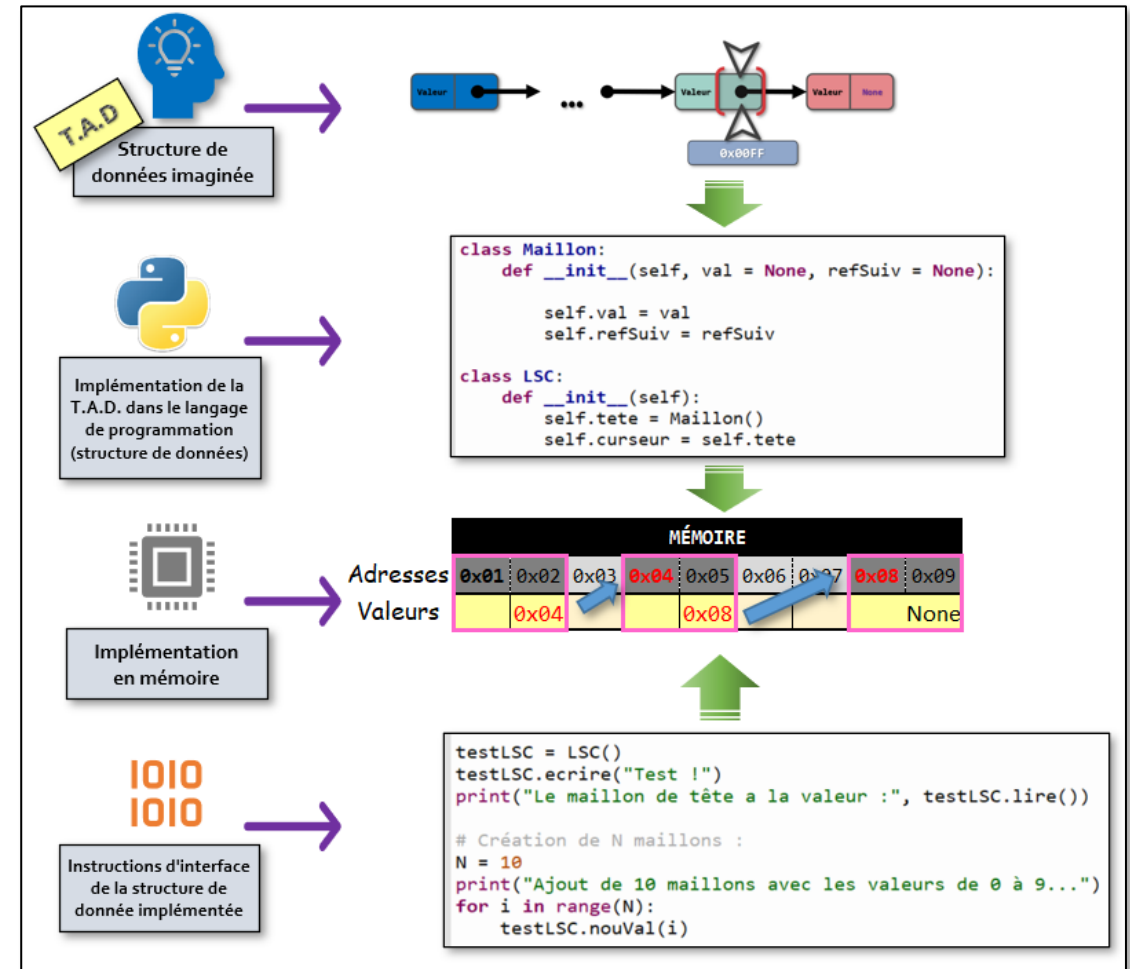
- Pour l'implémentation de la **T.A.D.** liste chaînée, nous utiliserons des classes de la **P.O.O.** (**P**rogrammation **O**rientée **O**bjet).



INTRODUCTION

- Les listes chaînées :

- Ainsi les listes chaînées sont constituées de cellules/maillons **NON CONTIGUS** en mémoire.
- La taille des données dans chaque cellules/maillons est libre et peut varier d'une cellule/maillon à l'autre.



INTRODUCTION

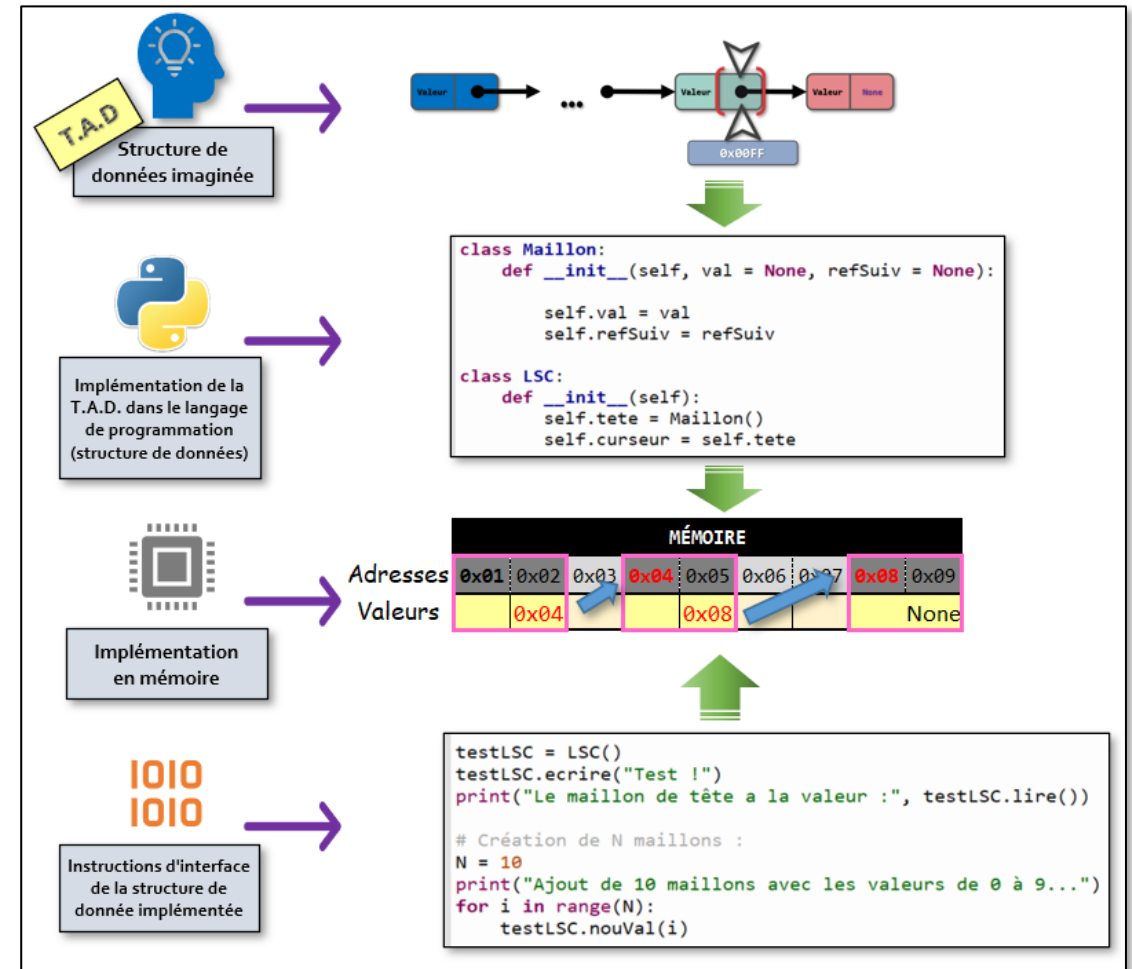
- Les listes chaînées :

- Avantage :

- On peut très facilement supprimer, insérer, rajouter des cellules/maillons. C'est une structure **DYNAMIQUE**

- Inconvénient :

- L'accès aux données se fait obligatoirement **SEQUENTIELLEMENT** (on doit parcourir toutes les cellules précédentes à la cellule ciblée).





- Et les listes de Python ?

- Les listes de **Python ne sont pas** des listes chaînées, mais des tableaux rendus dynamiques (le moteur d'exécution se charge des processus d'allocation des espaces mémoire pour nous).
- L'interface est proche de celle des tableaux d'ailleurs (accès par index), utilisation des délimiteurs `[]` pour définir la liste/tableau et accéder aux valeurs indexées

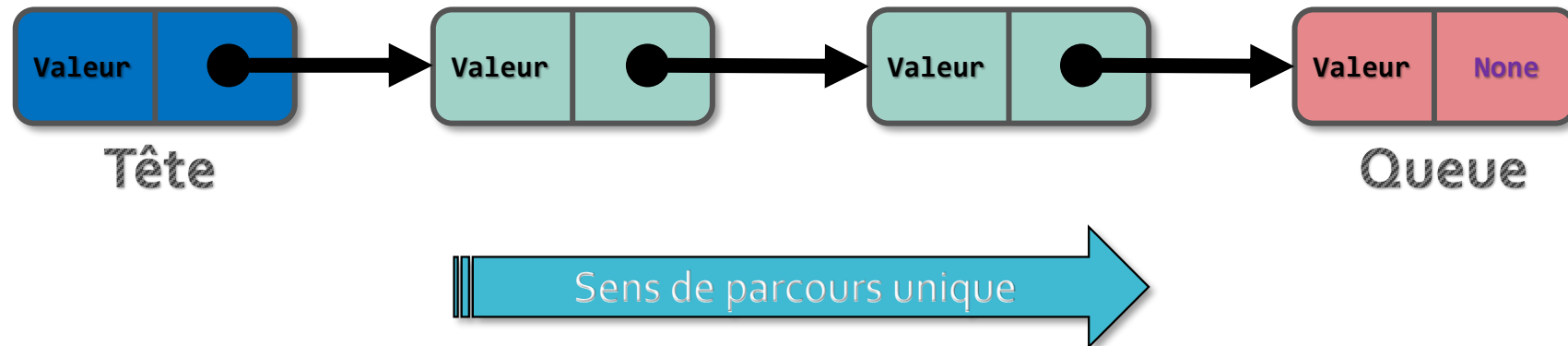
TYPOLOGIE DES LISTES CHAÎNÉES

Liste simplement chaînée / doublement chaînée /
circulaire

TPOLOGIE DES LISTES CHÂÎNÉES

- La liste simplement chaînée :

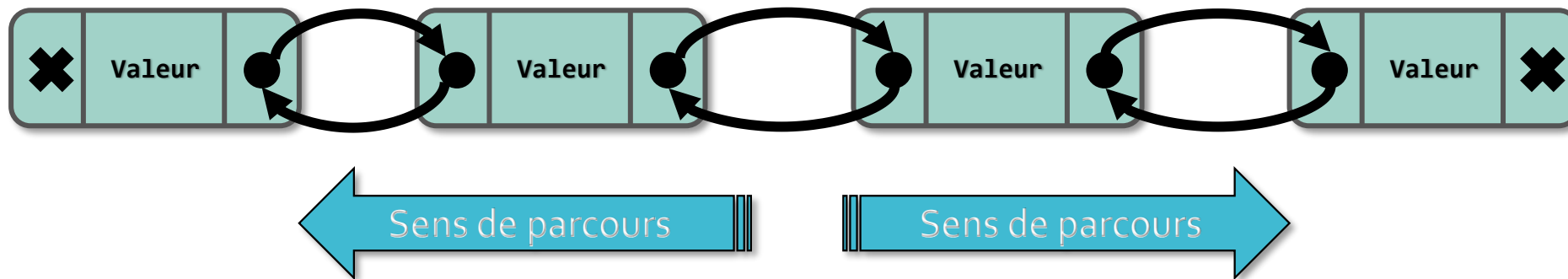
- C'est la chaîne que nous avons découvert précédemment et que nous allons implémenter.
- Dans cette chaîne le parcours ne peut se faire que dans un sens.



TPOLOGIE DES LISTES CHAÎNÉES

- La liste doublement chaînée :

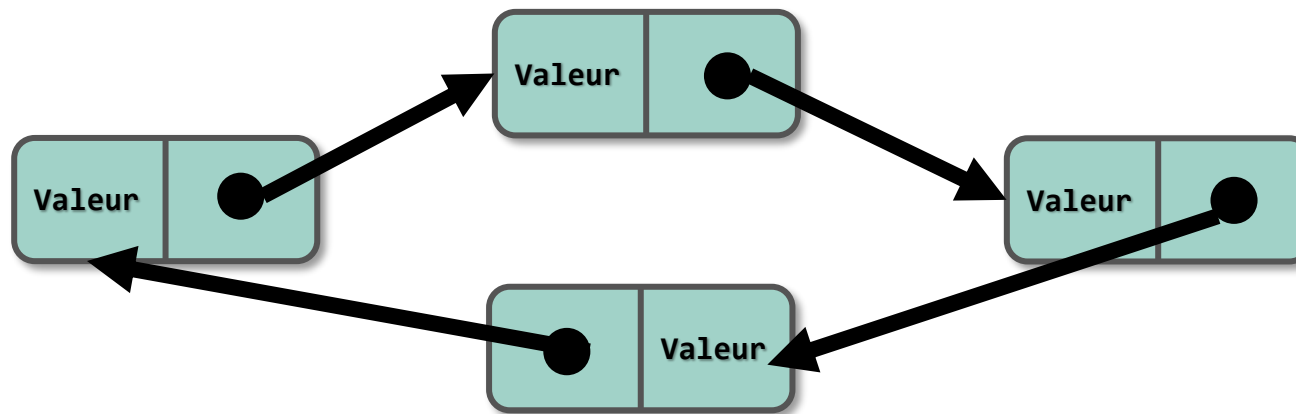
- Les maillons disposent ici à la fois de l'adresse du maillon suivant, mais aussi celui du précédent.
- Ainsi la chaîne peut être parcourue dans les deux sens.



TPOLOGIE DES LISTES CHAÎNÉES

- **La liste circulaire :**

- Ici la chaîne se referme sur elle-même.
- La chaîne peut être simplement ou doublement chaînée.
- Des précautions doivent être prises durant un parcours pour ne tomber dans une boucle infinie.



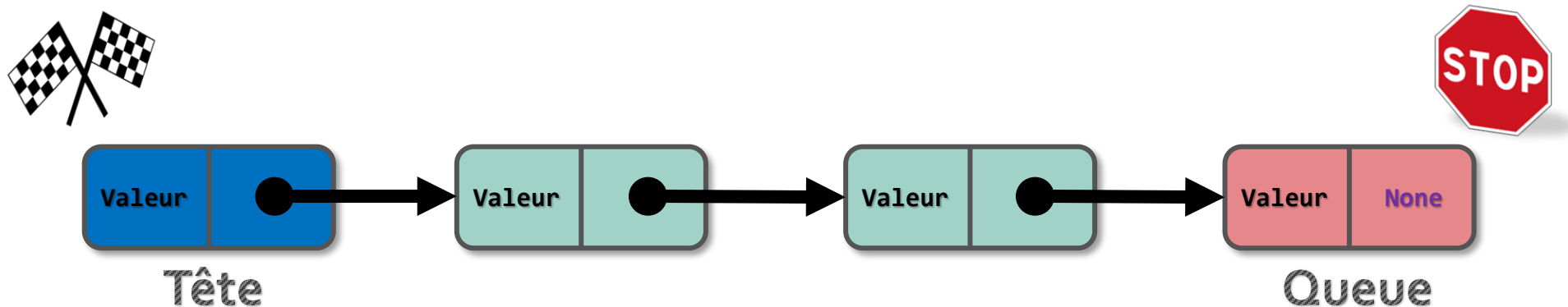
SPECIFICATION DU TAD LISTE CHAÎNÉES

Liens entre les données et primitives

SPÉCIFICATION DU TAD LISTE CHAÎNÉE

- Description générale de la T.A.D. liste chaînée :

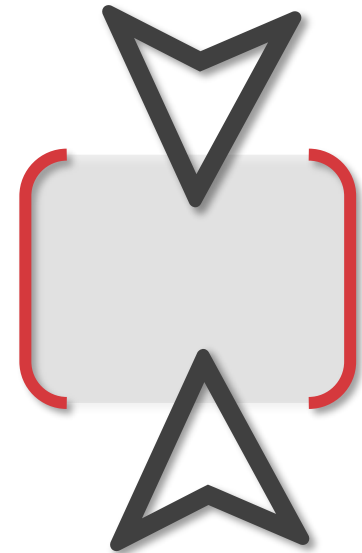
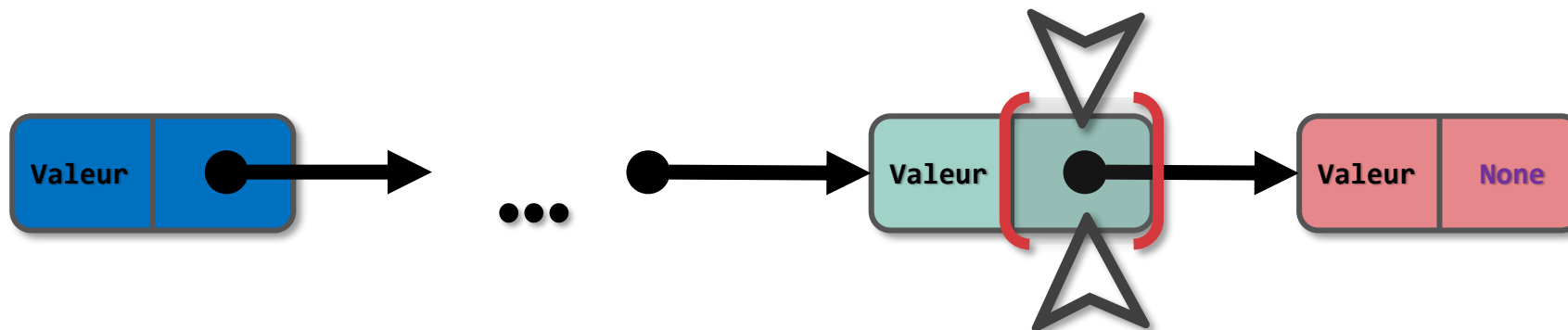
- Comme vu précédemment, les listes chaînées sont constituées de cellules/maillons.
- Chaque maillon comporte une donnée à stocker ET l'adresse du maillon suivant de la chaîne.
- Le 1^{er} maillon est appelé TÊTE et le dernier QUEUE.



SPÉCIFICATION DU TAD LISTE CHAÎNÉE

- Le curseur :

- Pour pouvoir « naviguer » dans notre LSC, nous avons besoin de mémoriser notre position dans la LSC.
- Le curseur opère cette fonction en assurant le « marquage » du maillon couramment occupé.



SPÉCIFICATION DU TAD LISTE CHAÎNÉE

- Primitives du T.A.D. liste chaînée :

- **nouVal** : Rajouter un nouveau maillon en queue avec une valeur.
- **lire** : Retourner la valeur contenu dans le maillon courant.
- **ecrire** : Ecrire une valeur dans le maillon courant.
- **nouVal** : Rajouter un nouveau maillon en queue avec une valeur.
- **avancer** : Pointer le « curseur » sur le maillon suivant.
- **fin** : Retourne « True » si le maillon courant est le maillon de queue.
- **surDebut** : Place le curseur sur le maillon de tête.
- **surFin** : Place le curseur sur le maillon de queue.
- **insMaillon** : Insère un nouveau maillon après le maillon courant.
- **supMaillon** : Supprime le maillon courant.

IMPLÉMENTATION DE LA LSC

Création des classes Cellule & Liste

IMPLÉMENTATION D'UNE LSC

- Implémentation :

- Nous allons implémenter 2 classes :
 - Une classe **Maillon** permettant d'instancier les différents maillons de tout objet de type LSC.
 - Une classe **LSC** permettant d'instancier nos listes chaînées. La classe **LSC** utilisera elle-même la classe **Maillon** comme structure de données de composition. Cette classe aura également un attribut pour assurer le rôle de **curseur**.

IMPLÉMENTATION D'UNE LSC

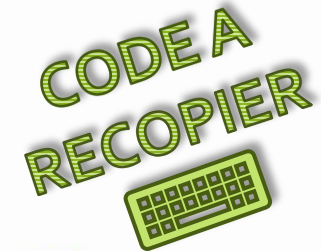
- La classe Maillon :

- Cette classe est assez rudimentaire, elle comporte 2 attributs permettant la mémorisation de la valeur à stocker et l'adresse du maillon suivant.

```
class Maillon:
    """Classe définissant les maillons d'une liste simplement chaînée."""

    def __init__(self, val = None, refSuiv = None):
        """Le constructeur d'un objet maillon va instancier les champs contenant la valeur
        à mémoriser ainsi que la référence pour atteindre le maillon suivant.
        Attention, le maillon de queue à son attribut <refSuiv> qui reste avec la valeur None."""

        self.val = val                # Valeur mémorisée par le maillon.
        self.refSuiv = refSuiv        # Variable contenant la référence de l'objet maillon suivant de la LSC.
```



IMPLÉMENTATION D'UNE LSC

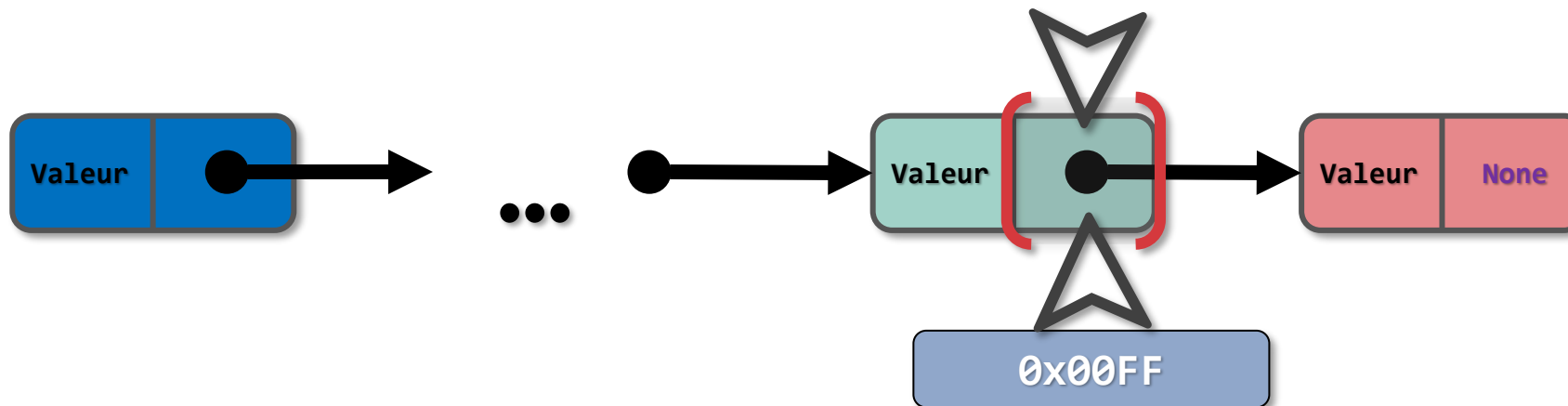
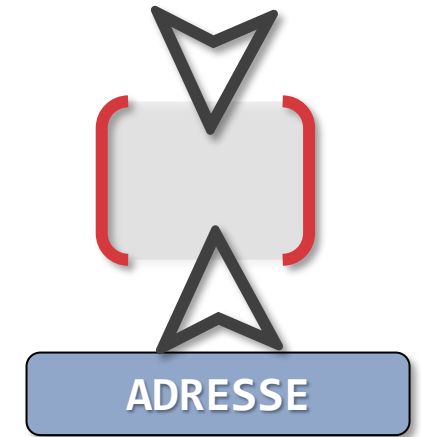
- **La classe LSC :**

- La classe LSC ne contient que 2 attributs ! Par contre elle est susceptible d'accueillir de nombreuses méthodes afin d'enrichir l'interface d'utilisation.
- Le 1^{er} attribut est le maillon de tête.
- Le second attribut est le « curseur ».

IMPLÉMENTATION D'UNE LSC

- **Le curseur :**

- Le curseur est le second attribut de notre LSC.
- Cet attribut mémorise l'adresse mémoire/référence du maillon couramment occupé dans la liste chaînée concernée.
- Sans le curseur, il n'est pas possible de naviguer/déplacer dans notre LSC.



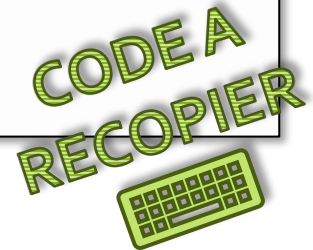
IMPLÉMENTATION D'UNE LSC

- Le code d'implémentation fondamentale de la LSC :
 - Notre classe LSC devra repose sur le code fondamental suivant :

```
class LSC:
    """Classe qui va définir notre LSC, elle contiendra donc la référence sur le maillon de tête
    de la liste chaînée. Le maillon de tête étant instancié par le constructeur de la classe LSC.
    L'ensemble des méthodes liées à la structure LSC sont incluses dans cette classe."""

    def __init__(self, val = None):
        """Le constructeur initialise la liste LSC en instanciant son maillon de tête. On initialise
        également le curseur qui permet de mémoriser notre position courante dans la LSC."""

        self.tete = Maillon()
        self.curseur = self.tete    # On initialise le curseur sur la tête.
```



IMPLÉMENTATION D'UNE LSC

- **Implémentation de quelques méthodes :**

- Avec le code précédent nous pouvons instancier un objet de classe LSC.
- Cependant notre objet est limité, on ne peut pas lui rajouter de valeur et nous ne pouvons pas avancer de maillon en maillon.
- Nous allons donc lui adjoindre quelques méthodes nous permettant de mener des tests.

IMPLÉMENTATION D'UNE LSC

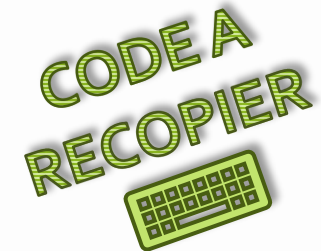
- La méthode : **nouVal(self, val = None)**:
 - Cette méthode va nous permettre d'enrichir nos instances avec de nouveaux maillons.
 - Il faut procéder en 3 étapes :
 1. **Créer un nouveau maillon**
 2. **Atteindre le maillon de queue de la LSC**
 3. **Renseigner la référence de notre nouveau maillon dans l'ex futur maillon de queue afin que notre nouveau maillon prenne ce rôle.**

IMPLÉMENTATION D'UNE LSC

- La méthode : **nouVal(self, val = None)**:

- Copier le code proposé ci-dessous :

```
def nouVal(self, val = None):  
    """Ajoute un maillon en queue de file et renseigne dans le maillon la valeur passée en  
    argument."""  
  
    # Création du nouveau maillon  
    nouveauMaillon = Maillon(val)  
  
    # Positionnement du curseur sur le maillon de queue :  
    while self.curseur.refSuiv != None:  
        self.curseur = self.curseur.refSuiv  
  
    # Inscription de la référence du nouveau maillon dans l'ancien maillon de queue :  
    self.curseur.refSuiv = nouveauMaillon
```

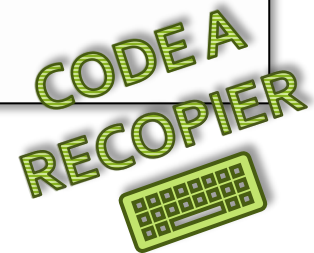


IMPLÉMENTATION D'UNE LSC

- La méthode : `lire(self)`:

- Certainement la méthode la plus simple, elle se contente de retourner la valeur du maillon curseur.

```
def lire(self):  
    """Retourne la valeur dans le maillon courant."""  
    return self.curseur.val
```

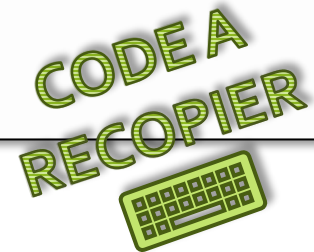


IMPLÉMENTATION D'UNE LSC

- La méthode : **ecrire(self, val):**

- Ecrire est à peine plus compliqué que de lire la valeur d'un maillon !

```
def écrire(self, val):  
    """Inscrit la valeur donnée en argument dans le maillon  
    courant."""  
    self.curseur.val = val
```



IMPLÉMENTATION D'UNE LSC

- **Se déplacer dans la LSC :**

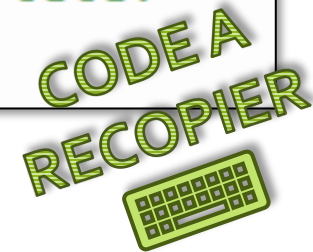
- Nous pouvons maintenant générer une LSC de plusieurs maillons.
- Nous pouvons lire et/ou écrire dans un maillon.
- MAIS, nous sommes actuellement bloqué sur le maillon de queue !!!!
- Nous allons à la page suivante créer la méthode nous permettant d'atteindre directement la tête de la LSC.

IMPLÉMENTATION D'UNE LSC

- La méthode : **surDebut(self)**:

- Pour atteindre le maillon de tête cela n'est pas compliqué car on a mémorisé la référence de maillon dans l'attribut **tête**.
- Il suffit de renseigner le curseur avec cette référence.

```
def surDebut(self):  
    """Place le curseur sur le maillon de tête."""  
    self.curseur = self.tete
```



IMPLÉMENTATION D'UNE LSC

- La méthode : **avancer(self)** :

- Il faut maintenant pouvoir nous déplacer de maillon en maillon de la LSC.
- Pour ce faire on implémente la méthode avancer.
- Observez bien la façon de procéder car on retrouvera cela dans d'autres méthodes.

```
def avancer(self):  
    """Place le curseur sur le maillon suivant."""  
    if self.curseur.refSuiv != None:  
        self.curseur = self.curseur.refSuiv
```

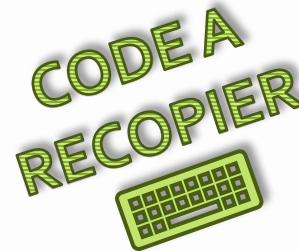


IMPLÉMENTATION D'UNE LSC

- 1^{ère} série de tests :

- Nous avons maintenant une implémentation minimal pour réaliser nos premiers essais.
- Procéder aux tests proposés ci-dessous :

```
# JEU DE TESTS :  
# _____  
  
# Instanciation d'une LSC :  
testLSC = LSC()  
  
# Ecriture d'une valeur :  
testLSC.ecrire("Hello !")  
  
# Lecture d'une valeur :  
print("Le maillon de tête a la valeur :", testLSC.lire())
```



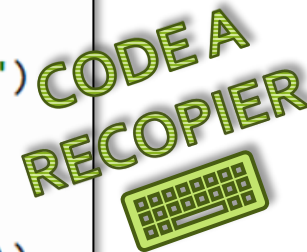
```
>>> %Run '01-LSC NSI.py'  
Le maillon de tête a la valeur : Hello !
```

IMPLÉMENTATION D'UNE LSC

- 1^{ère} série de tests :

- Rajoutons une série de maillons, puis vérifions que nous pouvons parcourir la LSC.

```
# Création de N maillons :  
N = 10  
print("Ajout de 10 maillons avec les valeurs de 0 à 9...")  
for i in range(N):  
    testLSC.nouVal(i)  
  
print("Lecture de la position courante :", testLSC.lire())
```



```
>>> %Run '01-LSC NSI.py'  
Ajout de 10 maillons avec les valeurs de 0 à 9...  
Lecture de la position courante : 8
```

IMPLÉMENTATION D'UNE LSC

- 1^{ère} série de tests :

- Nous allons maintenant retourner au début de la liste et lire tous les maillons.

```
print("Aller à la tête puis parcourir et lire les valeurs de tous les maillons :")
testLSC.surDebut()
fin = False
while fin != True:
    print(testLSC.lire(), end = "")
    if testLSC.curseur.refSuiv != None:
        testLSC.avancer()
        print(" / ", end = "")
    else:
        fin = True
        print()
```

CODE À
RECOPIER



```
>>> %Run '01-LSC NSI.py'
```

```
Ajout de 10 maillons avec les valeurs de 0 à 9...
Lecture de la position courante : 8
Aller à la tête puis parcourir et lire les valeurs de tous les maillons :
Hello ! / 0 / 1 / 2 / 3 / 4 / 5 / 6 / 7 / 8 / 9
```

IMPLÉMENTATION D'UNE LSC

• Amélioration du constructeur :

- La saisie est de plusieurs valeurs est pour le moment un peu lourde.
- On vous propose d'améliorer notre constructeur de LSC afin qu'il puisse « importer » des valeurs contenues dans une liste ou un tuple.
- Mettez à jour votre constructeur avec le code proposé.

```
def __init__(self, val = None):  
    """Le constructeur initialise la liste LSC en instanciant son maillon de tête. On initialise  
    également le curseur qui permet de mémoriser notre position courante dans la LSC."""  
  
    listeImport = []  
    # Si l'argument val n'est pas un itérable, on le converti en liste :  
    if type(val) != list and type(val) != tuple:  
        listeImport.append(val)  
    else:  
        listeImport = val  
    self.tete = Maillon(listeImport.pop(0)) # Initialisation de la tête.  
    self.curseur = self.tete               # Initialisation du curseur sur la tête.  
    for donne in listeImport:  
        self.curseur.refSuiv = Maillon(donne)  
        self.curseur = self.curseur.refSuiv
```



IMPLÉMENTATION D'UNE LSC

- **Amélioration du constructeur :**

- On peut maintenant procéder à l'instanciation d'un objet LSC comme ci-dessous :

```
lscSimple = LSC("1 valeur")  
lsc_A_PartirListe = LSC([12, 5, 9, 3, "yes"])  
lscSansValInitiale : LSC()
```

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Connaître le contenu de objet LSC reste pénible, nous sommes obligés de créer une boucle de parcours.
- **Implémenter** une méthode de type **__str__(self)** à la classe LSC qui nous permet de générer une chaîne de caractères présentant l'ensemble des valeurs contenues dans notre LSC.
- Le résultat attendu est le suivant :

```
importLSC = LSC([10, 11, 12, 13, 14, 15])  
print("La LSC à partir de l'import :", importLSC)
```

```
La LSC à partir de l'import : 10 -> 11 -> 12 -> 13 -> 14 -> 15
```

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def fin(self):  
    """Retourne True si curseur est sur le maillon de queue."""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def surFin(self):  
    """Place le curseur sur le maillon de queue."""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def insMaillon(self, val = None):  
    """Insertion d'un nouveau maillon entre le maillon courant  
    et son actuel voisin."""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def supMaillon(self):  
    """Suppression du maillon courant."""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def chercherVal(self, val):  
    """Positionne le curseur sur le 1er maillon contenant la valeur cherchée en partant  
    du maillon de tête. Le méthode retourne la référence du maillon. Si la valeur n'est  
    pas trouvée, le curseur se retrouve sur le maillon de queue et retourne None"""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def fusionner(self, lsc):  
    """Ajoute la liste chaînée lsc à la fin de la lsc sur laquelle on applique  
    la méthode."""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def couper(self):  
    """Coupe la liste à partir de son curseur et retourne en la référence de la LSC  
    constituée de la partie coupée."""
```

Code à compléter

IMPLÉMENTATION D'UNE LSC

- **TRAVAIL A FAIRE :**

- Implémenter l'ensemble des méthodes décrites ci-après :

```
def inserer(self, lsc):  
    """Insère sur la liste recevant la méthode la liste donnée en argument. Le point  
    d'insertion se fait juste après le curseur."""
```

Code à compléter



FIN